

Tracks A/B/C — Input Files (with provenance)

This note lists **which input files each Track A script reads** (Stage A, Stage B, and the helper scripts), and also includes **Track B and Track C input provenance** in the same document.

Scope: this is an *engineering-facing* inventory of inputs/outputs (frames, labels, manifests, weights, summaries). For each stage, it explains **what each artifact contains** and how it is consumed downstream.

0.5) What the core dataset artifacts contain (using `ego4d_samples/`)

This repo generally works with the full Ego4D STA JSONs. To avoid opening large files, the folder `ego4d_samples/` provides small “CSV-wrapped JSON” samples that mirror the same field structure.

STA annotation rows (sample schema)

- Sample files:
 - `ego4d_samples/v2/annotations/fho_sta_train_json.csv`
 - `ego4d_samples/v2/annotations/fho_sta_val_json.csv`
 - `ego4d_samples/v2/annotations/fho_sta_test_unannotated_json.csv`
- Each row (train/val) is one **decision frame** example and typically includes:
 - Identifiers: `uid` (often `${video_uid}_${frame}`), `video_uid` , `clip_uid` , `clip_id`
 - Decision frame index: `frame` (video frame), and `clip_frame`
 - Timing spans: `action*_sec|frame` , `action_clip*_sec|frame` , plus `interval_*` and `clip_parent_*`
 - Supervision list: `objects: [{box: [x1,y1,x2,y2], verb_category_id, noun_category_id, time_to_contact}]`
 - `box` is in pixel coordinates of the current frame
 - `time_to_contact` is in seconds
- Test-unannotated rows contain only identifiers (no `objects`): `uid, frame, clip_id, clip_uid, clip_frame, video_uid` .

These same fields are what the label builders/manifests are derived from.

0.6) Artifact catalog — what each *input/output file* contains

This section is a quick “what’s inside” catalog for the concrete artifacts referenced later.

Shared / upstream artifacts

- **Extracted frame JPEGs** (`local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`)
 - **Contains:** the decoded RGB frame image at the given frame index (typically 540p in this project).

- **Used by:** Track A Stage A inference/oracle conversion; Track A Stage B cropping; Track B/C feature extraction and CLIP re-ranking.
- **Frame lists** (`.../tmp_frame_lists/{clips|videos}/<uid>.txt`)
 - **Contains:** one integer per line (frame indices to extract/process for that uid).
 - **Used by:** the extractor when running in “from_lists” mode.
- **UID lists** (`sta_uids.txt` , `sta_clip_uids.txt`)
 - **Contains:** one uid per line (video_uids or clip_uids).
 - **Used by:** extraction + list splitting/resume logic.
- **YOLO label files**

(`local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt`)
 - **Contains:** YOLO-format rows per object: `cls cx cy w h` where `(cx,cy,w,h)` are normalized to image width/height.
 - **Role in this repo:** can serve as (a) oracle proposals for Track A, and/or (b) ground truth for Recall@K evaluation.
- **Head manifests** (`local_extraction/v2/manifests/head_{train,val}_{clip,video}.json`)
 - **Contains:** JSON list of per-(uid,frame) supervision records derived from STA annotations.
 - **Typical fields:**
 - `image` (path to the local frame JPG)
 - `gt_box` (pixel bbox)
 - `noun_id` , `verb_id` , `ttc` (seconds)
 - **Used by:** Track A Stage B to attach semantic labels and define positive/negative candidates.
- **Local “org annotation” mirrors** (`local_extraction/v2/org_annotations/*.json`)
 - **Contains:** local copies of Ego4D annotation JSONs and taxonomies (e.g., `fho_sta_val_height-540.json` , `fho_main.json` , narration taxonomies).
 - **Used by:** evaluation utilities that need noun/verb label text (e.g., CLIP prompt construction).
- **Hotspot priors** (`local_extraction/v2/hotspot_priors_*.json`)
 - **Contains:** a simple lookup table from `(noun_id, verb_id)` → prior value.
 - **Observed structure in this repo:**
 - `pairs` : dict keyed by the string `"<noun_id>,<verb_id>"` mapped to a float (often log-odds)
 - `default` : float fallback used when a pair is unseen
 - **Used by:** Track B/C scoring as an optional prior added to the final candidate score.
- **YOLO weights** (`*.pt`)
 - **Contains:** Ultralytics YOLO model weights.
 - **Used by:** Track A Stage A when proposal mode is YOLO.
- **Track configs** (`local_extraction/configs/trackA.yaml` , `trackB.yaml` , `trackC.yaml`)

- **Contains:** path roots + hyperparameters (K, thresholds), model settings, caching roots, and evaluation toggles.

Track A artifacts

- **Stage A proposals** (`candidates.jsonl`)
 - **Contains:** one JSON per processed frame: `uid` , `frame` , and `boxes` (each with pixel bbox + confidence + class id).
 - **Role:** proposal set passed into Stage B.
- **Stage A run summary** (`summary.json`)
 - **Contains:** run configuration (mode, K, thresholds, roots) and optional recall-style stats.
- **Stage B crop manifest** (`manifest.jsonl`)
 - **Contains:** one JSON per candidate crop linking `(uid, frame, roi_idx)` → `crop_path` plus the candidate bbox/score.
 - **If supervision enabled:** adds `hit` , `iou` , `is_positive` , and semantic labels (`noun_id` , `verb_id` , `ttc`).
 - **Role:** audit/debug file; also a convenient flat table for analysis.
- **Stage B crops** (`crops/<uid>/<frame>_<roi_idx>.jpg`)
 - **Contains:** cropped candidate regions (JPEG) produced from the original frames.
- **Track B training/validation manifests (from Stage B)** (`head_train.jsonl` , `head_val.jsonl`)
 - **Contains:** one JSON per candidate with fields like `image_path` , `candidate_box` , `candidate_conf` , semantic labels, and `is_positive` .
 - **Role:** primary dataset files for Track B/C.
- **Stage B run summary** (`summary.json`)
 - **Contains:** references to input candidates/head manifests and computed recall metrics.
- **Stage B preview images** (`overlay_preview.jpg` , `comparison_preview.jpg`)
 - **Contains:** visualization snapshots used for quick sanity checks (not required for training).

Track B artifacts

- **Model checkpoints** (`local_extraction/runs/Track_B/checkpoints/*.pt`)
 - **Contains:** PyTorch state dict(s) for the Track B fusion head and any projection layers.
- **Track B run logs** (`local_extraction/runs/Track_B/run_log.json`)
 - **Contains:** full provenance of a run:
 - `git` (commit/branch/dirty), `system` (python/torch/cuda), `config` (paths + toggles)
 - `metrics` (mAP/MAE plus the thesis “top-5” metric variants and per-noun stats)

- **Evaluation metric dumps** (`local_extraction/runs/Track_B/metrics/metrics_val_*.json` and `*_summary.json`)
 - **Contains:** scalar evaluation metrics and often the training/eval config used.
 - **Includes (commonly):** `N_mAP` , `Nv_mAP` , `N_delta_mAP` , `All_mAP` and the corresponding `*_top5_acc` and `*_top5_mAP` fields.
- **Prediction exports**
 - (`local_extraction/runs/Track_B/predictions/predictions_val_*.jsonl` and `.csv`)
 - **Contains:** per-candidate prediction rows aligned to the validation set.
 - **Common fields:** uid, frame path, candidate bbox, GT vs predicted label, final score components (hotspot/CLIP), TTC prediction/GT/error, and GT vs predicted noun/verb (and optional TTC bins).
 - **Role:** error analysis and plotting.

Track C artifacts

- **Track C run logs** (`local_extraction/runs/Track_C/run_log.json`)
 - **Contains:** run provenance + nested configs (`eval_config` , `rgtp_config` , `instrumentation_config`) and final `metrics` .
- **Track C metric dumps** (`local_extraction/runs/Track_C/metrics/trackC_val_rate*_.json` and `*_summary.json`)
 - **Contains:** semantic metrics like Track B plus pruning/instrumentation fields.
 - **Common pruning fields:** `rgtp_enabled` , `rgtp_rate_request` , `rgtp_mean_fraction_pruned` .
 - **Common runtime fields:** latency percentiles, throughput, and optional FLOPs/benchmark times.

0) How each input is created (upstream provenance)

This section explains **how each Track A input artifact is produced** elsewhere in the pipeline (or manually), so you can reproduce the inputs from scratch.

- **Frame images** (`.../<uid>/<frame:07d>.jpg`)
 - **Created by:** the frame extraction runner `local_extraction/extraction/ego4d_resume_fast_extract.py`
 - **Source data:** Ego4D clip/video MP4s under your Ego4D data root (e.g., `.../ego4d_data/v2/clips_540/clips/<uid>.mp4` or `.../ego4d_data/v2/video_540ss/<uid>.mp4`).
 - **Which frames get extracted:** typically driven by per-uid “frame list” text files under `.../ego4d_data/v2/tmp_frame_lists/{clips|videos}/` , where each file enumerates frame indices to extract.
 - **Where frame lists come from:** generated from STA annotations by `local_extraction/labels/sta_builder_local.py` when you enable `BUILD_CLIP_LISTS` and/or `BUILD_VIDEO_LISTS` .

- **Important path note (Drive vs local):** `sta_builder_local.py` can write these lists either to your repo (default `OUTPUT_ROOT=<repo>/local_extraction`) or to Drive (if you set `OUTPUT_ROOT` to your Drive Ego4D root). The extractor reads lists from `EGO4D_ROOT` , so the simplest “no-copy” setup is: set `OUTPUT_ROOT` to the same root as `EGO4D_ROOT` when generating lists.
- **Output location:** `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg` (or the configured output root/version).
- **Drive-side inputs used to create Track-A prerequisites**
 - **STA annotations (full-res or height-540):**
 - **Location (typical):** `H:/My Drive/ego4d_data/<version>/annotations/` or `H:/My Drive/ego4d_data/<version>/annotations_540ss/`
 - **Key files (v2):** `fho_sta_train.json` / `fho_sta_val.json` or `fho_sta_train_height-540.json` / `fho_sta_val_height-540.json`
 - **Where they come from:** official Ego4D dataset downloads via the Ego4D tooling (this repo’s docs include examples of downloading `annotations_540ss`).
 - **When/where they are used in this repo:**
 - **Build UID lists + frame lists:** `local_extraction/labels/sta_builder_local.py`
 - **Generate YOLO GT labels:** `local_extraction/labels/sta_builder_local.py` (optional) and `local_extraction/pipelines/local_sta_pipeline.py` (`labels` command)
 - **Generate semantic head manifests:** `local_extraction/labels/sta_builder_local.py`
 - **Evaluation-time label mapping (e.g., CLIP noun prompts):** Track B/Track C eval defaults can read a height-540 STA JSON for noun label text.
 - **tmp_frame_lists on Drive (if present):**
 - **Location (typical):** `H:/My Drive/ego4d_data/<version>/tmp_frame_lists/{clips|videos}/`
 - **What it is:** per-UID text files of frame indices (one integer per line).
 - **Where it comes from:** usually generated from annotations by `sta_builder_local.py` (or an equivalent list builder), then consumed by the extractor.
- **Local annotation mirrors (sometimes used instead of Drive paths)**
 - **Location (typical):** `local_extraction/v2/org_annotations/`
 - **What it is:** a local copy/mirror of selected annotation JSONs (often including STA height-540 JSONs).
 - **Why it exists:** lets local evaluation/analysis scripts run without requiring the Drive `EGO4D_ROOT` layout.
 - **Typical usage:** Track B/Track C evaluation utilities can read `local_extraction/v2/org_annotations/fho_sta_val_height-540.json` to build noun-label text/prompt mappings for CLIP re-ranking.
- **Local outputs produced by `sta_builder_local.py` (and used downstream)**
 - **UID files:** `sta_uids.txt` , `sta_clip_uids.txt`
 - **Created by:** `local_extraction/labels/sta_builder_local.py`
 - **How:** parses STA annotations and writes the unique video_uids / clip_uids.

- **Used by:** the frame extractor `local_extraction/extraction/ego4d_resume_fast_extract.py` (directly or via part files).
 - **Frame lists:** `local_extraction/<version>/tmp_frame_lists/{videos,clips}/*.txt`
 - **Created by:** `sta_builder_local.py` with `BUILD_VIDEO_LISTS=True` / `BUILD_CLIP_LISTS=True`.
 - **Used by:** `ego4d_resume_fast_extract.py` when `EXTRACTION_MODE="from_lists"`.
 - **Optional YOLO labels:** `local_extraction/<version>/yolo_labels_540/{videos,clips}/<uid>/<frame:07d>.txt`
 - **Created by:** `sta_builder_local.py` when `GENERATE_YOLO_LABELS=True` (or by the label generator inside `local_sta_pipeline.py`).
 - **Used by:** Track A Stage A (oracle proposals) and Track A Stage B (Recall@K evaluation).
 - **Optional head manifests:** `local_extraction/<version>/manifests/head_{train,val}_{video,clip}.json`
 - **Created by:** `sta_builder_local.py` when `GENERATE_HEAD_MANIFESTS=True`.
 - **Used by:** Track A Stage B to attach noun/verb/TTC supervision to candidates.
- **YOLO-format label files** (`.../{clips|videos}/<uid>/<frame:07d>.txt`)
 - **Created by (option A):** `local_extraction/labels/sta_builder_local.py` when `GENERATE_YOLO_LABELS=True`.
 - **Created by (option B):** the “labels” command inside `local_extraction/pipelines/local_sta_pipeline.py` (idempotent: creates missing labels and skips existing ones).
 - **Source data:** Ego4D-STA annotation JSONs (either height-540 scaled or full-res), typically under `.../ego4d_data/v2/annotations_540ss/` (for `*_height-540.json`) or `.../ego4d_data/v2/annotations/` (for full-res).
 - **What gets written:** each `.txt` contains normalized YOLO rows `cls cx cy w h`.
 - Class policy depends on the builder settings (e.g., single-class vs noun-as-class).
 - If width/height is missing in the JSON record, the builder can infer it by probing the corresponding local frame image.
 - **Output location:** `local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt`.
- **Head manifests** (`head_train_clip.json` , `head_val_clip.json` , `head_train_video.json` , `head_val_video.json`)
 - **Created by:** `local_extraction/labels/sta_builder_local.py` when `GENERATE_HEAD_MANIFESTS=True` (also controllable via `EGO4D_GENERATE_HEAD_MANIFESTS=1`).
 - **Source data:** the same Ego4D-STA annotation JSONs.
 - **What gets written:** JSON lists, each item typically contains:
 - `image` (path to the corresponding local frame JPG)
 - `gt_box` (pixel box)
 - `noun_id` , `verb_id` , `ttc`
 - Optionally, one-object selection per (uid,frame) via “min TTC” policy.
 - **Output location:** `local_extraction/v2/manifests/`.

- **YOLO weights** (`*.pt`)
 - **Created by:** training/fine-tuning YOLO (Ultralytics) either from a base checkpoint or from scratch.
 - **Common sources in this repo:**
 - A baseline checkpoint `yolo8s.pt` (used as a default starting point for inference/training).
 - Fine-tuned checkpoints produced via the workflow documented in [local_extraction/toolkit_yolo/README.md](#):
 - Assemble a YOLO dataset locally from extracted frames + YOLO labels.
 - Package/upload to Colab.
 - Fine-tune in `Yolo_Ego4d.ipynb`.
 - Copy the resulting `runs/.../weights/best.pt` back into `local_extraction/toolkit_yolo/runs/.../`.
 - **Track A YAML config** (`local_extraction/configs/trackA.yaml`)
 - **Created by:** manual authoring (it's configuration, not generated).
 - **Purpose:** defines the roots for frames/labels/manifests/runs and Stage A/B hyperparameters (K, thresholds, crop settings, evaluation toggles).
 - **Stage A proposals** (`candidates.jsonl`)
 - **Created by:** Stage A itself, [local_extraction/trackA/trackA_stageA/trackA_stageA.py](#).
 - **How:** for each input frame, it either:
 - runs YOLO inference using a chosen `*.pt` weights file, or
 - converts YOLO label `.txt` boxes (oracle mode) then writes one JSON record per image into `candidates.jsonl`.
 - **Output location:** inside the Stage A run directory under `local_extraction/runs/Track_A/.../`.
 - **Stage B run artifacts used by helpers** (`summary.json` , `head_train.jsonl` , `head_val.jsonl` , `manifest.jsonl`)
 - **Created by:** Stage B itself, [local_extraction/trackA/trackA_stageB/trackA_stageB.py](#).
 - **How:** it reads `candidates.jsonl` and frame images, crops each ROI, and writes per-crop manifests; if head manifests exist it also writes consolidated `head_train.jsonl` / `head_val.jsonl` aligned to candidates.
 - **Used by:** `san_stageB.py` (inspection) and `trackA_plots.py` (plotting/summary).
-

1) Stage A — Proposal generation

`local_extraction/trackA/trackA_stageA/trackA_stageA.py`

Reads:

- **Frame images** (decision frames or all frames, depending on configuration):
 - Pattern: `.../<uid>/<frame:07d>.jpg`
 - Used to run either oracle conversion (needs image size) or YOLO inference.

Produces:

- **Stage A proposals:** `candidates.jsonl`
 - One JSON object per input image (one line per frame):
 - `uid` (clip/video uid), `frame` (frame index)
 - `boxes` : list of candidate detections; each item typically has:
 - `x1,y1,x2,y2` (pixel coords), `conf` (score), `cls` (class id)
 - Meaning: the *proposal set* for Track A (and upstream for Tracks B/C via Stage B manifests).
 - **Stage A run summary:** `summary.json`
 - Contains run metadata (mode, K, roots, YOLO config) and (if enabled) recall stats such as `recall_at_K`, `mean_best_iou`, and an IoU histogram.
 - **YOLO-format label files** (oracle mode only):
 - Pattern: `.../{clips|videos}/<uid>/<frame:07d>.txt`
 - Contents: YOLO normalized rows `cls cx cy w h`.
 - Used as *proposal boxes* (converted to pixel coords) when proposal mode is `oracle`.
 - **YOLO weights** (YOLO mode only):
 - File: a `*.pt` weights file (default `yolo8s.pt` unless overridden).
 - Used to run Ultralytics inference when proposal mode is `yolo`.
 - **YAML config:**
 - File: `local_extraction/configs/trackA.yaml`
 - Used to resolve frame roots, label roots, K, thresholds, and run settings.
-

2) Stage B — Crops + recall + semantic attachment

`local_extraction/trackA/trackA_stageB/trackA_stageB.py`

Reads:

- **Stage A proposals** (primary input):
 - File: `candidates.jsonl` (either explicitly provided or auto-detected as “latest run”).
 - Contains (per line): `{"uid":..., "frame":..., "boxes": [{"x1", "y1", "x2", "y2", "conf", "cls"}, ...]}`
 - Used to enumerate candidate boxes per decision frame.
- **Frame images** (to crop ROIs for each candidate):
 - Pattern: `.../<uid>/<frame:07d>.jpg`
 - Used to crop candidate regions; optionally resized.
- **YOLO-format label files** (only if recall evaluation is enabled):
 - Pattern: `.../{clips|videos}/<uid>/<frame:07d>.txt`

- Contents: YOLO normalized rows `cls cx cy w h`.
- Used to compute Recall@K and IoU statistics against the candidate boxes.
- **Head manifests** (optional semantic supervision source; if present):
 - Files (JSON lists):
 - `head_train_clip.json` or `head_train_video.json`
 - `head_val_clip.json` or `head_val_video.json`
 - Each item is expected to contain an `image` path and semantic fields such as `gt_box`, `noun_id`, `verb_id`, `ttc`.
 - Used to attach `is_positive`, `noun_id`, `verb_id`, `ttc` (and derived/optional TTC-bin) to the candidate set.
- **YAML config:**
 - File: `local_extraction/configs/trackA.yaml`
 - Used to resolve how to find candidates, whether to compute recall, which head manifests to use, and cropping settings.

Produces:

- **Per-candidate crop manifest:** `manifest.jsonl`
 - One line per candidate box (so file length $\approx$ total candidates across all frames).
 - Typical fields:
 - `uid`, `frame`, `roi_idx` (index within frame), `crop_path`
 - Candidate geometry/score: `x1,y1,x2,y2`, `conf`, `cls`
 - If labels/head supervision are enabled: `hit`, `iou`, `is_positive`, `noun_id`, `verb_id`, `ttc`
 - Meaning: a flat table that links a (uid, frame, candidate) to its crop image and (optional) supervision.
- **Crop images:** `crops/<uid>/<frame>_<roi_idx>.jpg`
 - JPEG crops extracted from the frame image using the candidate box.
 - Meaning: the actual visual input that Track B's head can consume if configured to use crops.
- **Train/val candidate manifests for Track B:** `head_train.jsonl`, `head_val.jsonl`
 - JSONL where each line is one candidate example with supervision:
 - `image_path` (full frame path) + `candidate_box` (pixel coords)
 - `candidate_conf`, `candidate_cls`
 - `noun_id`, `verb_id`, `ttc`, `is_positive`, and `iou`
 - Meaning: the *main dataset* consumed by Track B training/evaluation.
- **Stage B run summary:** `summary.json`
 - Run metadata plus recall metrics (if enabled) and references to which head manifests were used.

3) Sweeps and analysis helpers

`local_extraction/trackA/trackA_stageA/oracle_k_sweep.py`

Reads (indirectly, by launching Stage A and Stage B):

- Same inputs as:
 - `local_extraction/trackA/trackA_stageA/trackA_stageA.py` (frames, optional YOLO labels)
 - `local_extraction/trackA/trackA_stageB/trackA_stageB.py` (candidates, frames, optional labels and head manifests)

Reads (directly):

- **Stage B summary:**
 - File: `summary.json` in the latest Stage B run folder.
 - Used to extract recall metrics per K.

Note: this script defines internal paths for the Stage A/B scripts; if those constants don't match your current folder layout, it may need path adjustment, but its *input dependencies* remain as listed above.

`local_extraction/trackA/trackA_stageA/sweep_k_metrics.py`

Reads:

- **Frame images:**
 - Pattern: `.../<uid>/<frame:07d>.jpg`
- **YOLO-format label files** (used as ground truth):
 - Pattern: `.../{clips|videos}/<uid>/<frame:07d>.txt`
 - Contents: YOLO normalized rows `cls cx cy w h`.
- **YOLO weights:**
 - File: a `*.pt` weights file (path is hard-coded in this script's config block).

This script does *not* depend on Stage A/B outputs; it runs its own YOLO inference and computes recall-like statistics for multiple K values.

`local_extraction/trackA/trackA_stageB/san_stageB.py`

Reads (from the most recent Stage B run folder):

- `head_train.jsonl`
- `head_val.jsonl`
- `summary.json`
- `manifest.jsonl` (optional; used to pick a sample record)

May also read (for preview images):

- The underlying **frame image** referenced by a manifest record.

- The **crop image** referenced by `crop_path` in a manifest record.

Produces:

- Typically no new core artifacts; it is an inspection/visualization helper.

`local_extraction/trackA/trackA_plots.py`

Reads:

- A `summary.json` file (either a Stage B summary or a K-sweep summary JSON, depending on what you point it at).

This script is pure post-processing: it does not read frames, labels, candidates, or manifests.

Produces:

- Plots/figures (e.g., saved images) based on the summary metrics you point it at.

4) Track B — Inputs (and how each input is created)

Track B trains/evaluates the fusion head on top of Track A Stage B candidate manifests.

4.0) How each input is created (upstream provenance)

- **Frame images** (`.../<uid>/<frame:07d>.jpg`)
 - **Created by:** the frame extraction runner `local_extraction/extraction/ego4d_resume_fast_extract.py`
 - **Frame selection lists (optional but typical):** per-UID text files under `.../ego4d_data/v2/tmp_frame_lists/{clips|videos}/` .
 - **Where frame lists come from:** generated from STA annotations by `local_extraction/labels/sta_builder_local.py` when you enable `BUILD_CLIP_LISTS` / `BUILD_VIDEO_LISTS` .
- **Stage B candidate manifests for training/validation** (`head_train.jsonl` / `head_val.jsonl`)
 - **Created by:** Track A Stage B, `local_extraction/trackA/trackA_stageB/trackA_stageB.py`
 - **How:** merges Stage A proposals (`candidates.jsonl`) with optional semantic supervision from “head manifests” (noun/verb/TTC), and writes per-candidate rows.
 - **Upstream prerequisites:**
 - Stage A proposals created by `local_extraction/trackA/trackA_stageA/trackA_stageA.py`
 - Optional semantic supervision created by `local_extraction/labels/sta_builder_local.py` as `head_*_{clip|video}.json`
- **Base head manifests (semantic supervision source)** (`head_train_clip.json` , `head_val_clip.json` , `head_train_video.json` , `head_val_video.json`)
 - **Created by:** `local_extraction/labels/sta_builder_local.py` when `GENERATE_HEAD_MANIFESTS=True`

- **Used by:** Track A Stage B as the source of `noun_id` , `verb_id` , `ttc` and `gt_box` aligned to frames.
- **Track B checkpoint(s)** (`trackB_best.pt` , `trackB_final_*.pt` , etc.)
 - **Created by:** Track B training, [local_extraction/trackB/trackB_train_loader.py](#)
 - **Used by:** Track B evaluation [local_extraction/trackB/trackB_eval.py](#) and Track C pruning [local_extraction/trackC/trackC_pruning.py](#)
- **Optional pre-extracted ResNet18 tokens** (`v2/resnet18_tokens/<uid>/<frame>.pt`)
 - **Created by:** [local_extraction/trackB/extract_resnet_tokens.py](#)
 - **How:** runs the Track B tokenizer backbone on each extracted frame and saves grid tokens (typically `(49, 512)`) as `.pt` files.
 - **Used by:** Track B dataset/tokenizer when `data.tokens_root` is set in [local_extraction/configs/trackB.yaml](#).
- **Optional annotation-aligned video tensors** (`*.pt` tensors per (uid,frame))
 - **Created by:** [local_extraction/trackB/build_trackB_tensors.py](#)
 - **Purpose:** speeds up training/eval by precomputing the exact temporal window tensor used by the tokenizer.
- **VideoMAE encoder weights** (`runs/VideoMAE/videomae_ego_encoder.pt` by default)
 - **Created by:** VideoMAE pretraining code under [local_extraction/trackB/videomae/](#) (documented in [local_extraction/trackB/videomae_readme.md](#)).
 - **Used by:** Track B tokenizer when `model.tokenizer.video_backbone: videomae_ego` in [local_extraction/configs/trackB.yaml](#).
- **Hotspot priors JSON** (e.g., `hotspot_priors_*_logodds_min10.json`)
 - **Created by:** [local_extraction/trackB/build_hotspot_priors.py](#)
 - **How:** estimates $P(\text{next-active} \mid \text{noun,verb})$ (or log-odds) from Stage B manifests.
 - **Used by:** Track B evaluation as an optional score prior in [local_extraction/trackB/trackB_eval.py](#).
- **Org annotation JSON used for noun labels in CLIP re-ranking** (default: `local_extraction/v2/org_annotations/fho_sta_val_height-540.json`)
 - **Where it comes from:** copied/mirrored from the Ego4D STA annotation downloads into [local_extraction/v2/org_annotations/](#) (local convenience).
 - **Used by:** Track B evaluation to build noun text prompts/labels for CLIP re-ranking.
- **Track B YAML config** ([local_extraction/configs/trackB.yaml](#))
 - **Created by:** manual authoring (configuration).
 - **Purpose:** points Track B to frames/manifests/tokens/weights and defines model + training/eval settings.

4.1) Core Track B inputs (quick list)

- Training: `head_train.jsonl` (or a discoverable `head_train*.{json,jsonl}`)

- Validation: `head_val.jsonl` (or `head_val*.{json,jsonl}`)
- Frames: `extracted_frames/<uid>/<frame>.jpg`
- Optional speedups: `v2/resnet18_tokens/**` or prebuilt tensors
- Optional backbones/priors: `videomae_ego_encoder.pt` , hotspot priors JSON

4.2) What Track B outputs contain (common run artifacts)

Track B writes its outputs under `local_extraction/runs/Track_B/` .

- **Checkpoints** (`local_extraction/runs/Track_B/checkpoints/*.pt`)
 - PyTorch model weights for the fusion head (and any configured projectors).
 - Naming often encodes the monitored metric (e.g., `trackB_best_mAP_...pt`).
- **Checkpoint summaries** (`*_summary.json` , e.g., `trackB_best_summary.json`)
 - Records: which `.pt` was chosen as “best”, the monitored metric/value, and the training config used.
- **Validation metrics dumps** (`local_extraction/runs/Track_B/metrics/metrics_val_*.json` and `*_summary.json`)
 - Contains scalar metrics such as `mAP` , `ttc_mae_seconds` , and also the thesis metrics fields:
 - `N_mAP` , `Nv_mAP` , `N_delta_mAP` , `All_mAP`
 - `N_top5_acc` , `Nv_top5_acc` , `N_delta_top5_acc` , `All_top5_acc`
 - `N_top5_mAP` , `Nv_top5_mAP` , `N_delta_top5_mAP` , `All_top5_mAP`
 - Meaning: the evaluation report for one checkpoint/config on the validation manifest.
- **Prediction exports**

(`local_extraction/runs/Track_B/predictions/predictions_val_*.jsonl` and `.csv`)
 - Per-candidate predictions aligned with the validation manifest rows.
 - Intended for error analysis and plotting (confusions, per-noun breakdowns, etc.).

5) Track C — Inputs (and how each input is created)

Track C evaluates a Track B checkpoint with optional RGTP pruning (no training in Track C).

5.0) How each input is created (upstream provenance)

- **Track B checkpoint** (`trackB_best.pt` , `trackB_final_*.pt` , etc.)
 - **Created by:** Track B training, [local_extraction/trackB/trackB_train_loader.py](#)
 - **Used by:** Track C evaluation/pruning, [local_extraction/trackC/trackC_pruning.py](#)
- **Stage B validation manifest** (`head_val.jsonl` / `head_val.json` or `head_val_{clip|video}.json[1]`)
 - **Created by (preferred):** Track A Stage B, [local_extraction/trackA/trackA_stageB/trackA_stageB.py](#) as `head_val.jsonl` .

- **Created by (base alternative):** `local_extraction/labels/sta_builder_local.py` as `head_val_clip.json` / `head_val_video.json` .
- **Role:** provides candidate boxes and (optionally) noun/verb/TTC fields for semantic metrics.
- **Frame images** (`.../<uid>/<frame:07d>.jpg`)
 - **Created by:** the frame extraction runner `local_extraction/extraction/ego4d_resume_fast_extract.py`
 - **Frame lists (optional):** generated from annotations by `local_extraction/labels/sta_builder_local.py`.
- **Optional pre-extracted tokens/tensors** (config-driven)
 - **Created by:** Track B token/tensor builders:
 - ResNet18 grid tokens: `local_extraction/trackB/extract_resnet_tokens.py`
 - Annotation-aligned tensors: `local_extraction/trackB/build_trackB_tensors.py`
 - **Used by:** Track C through the shared Track B dataset/tokenizer logic (Track C imports Track B modules).
- **VideoMAE encoder weights** (`runs/VideoMAE/videomae_ego_encoder.pt` by default)
 - **Created by:** VideoMAE pretraining code under `local_extraction/trackB/videomae/` (documented in `local_extraction/trackB/videomae_readme.md`).
 - **Used by:** Track C when you run with `--video_backbone videomae_ego` and the config points to the weights.
- **Hotspot priors JSON** (optional)
 - **Created by:** `local_extraction/trackB/build_hotspot_priors.py`
 - **Used by:** Track C if hotspot priors are enabled.
- **Org annotation JSON for noun labels (CLIP re-ranking)** (optional)
 - **Provided/placed at:** `local_extraction/v2/org_annotations/fho_sta_val_height-540.json` (default used by Track B and Track C)
 - **Used by:** Track C if CLIP re-ranking is enabled.
- **Track C YAML config** (`local_extraction/configs/trackC.yaml`)
 - **Created by:** manual authoring (configuration).
 - **Purpose:** selects checkpoint/manifest discovery and sets RGTP + instrumentation parameters.

5.1) Core Track C inputs (quick list)

- Checkpoint: a Track B `.pt` checkpoint
- Validation candidates: `head_val*.{json,jsonl}`
- Frames: `extracted_frames/<uid>/<frame>.jpg`
- Optional: cached tokens/tensors, VideoMAE weights, hotspot priors, org annotations for CLIP

5.2) What Track C outputs contain (common run artifacts)

Track C writes its outputs under `local_extraction/runs/Track_C/` .

- **Metrics dumps** (`local_extraction/runs/Track_C/metrics/trackC_val_rate*_*.json` and `*_summary.json`)
 - Contains the same semantic metrics as Track B plus pruning/instrumentation:
 - `rgtp_enabled` , `rgtp_rate_request` , `rgtp_mean_fraction_pruned`
 - Latency/throughput stats such as `latency_ms_mean|median|p90|p95` , `throughput_*`
 - Optional compute stats like `head_flops` and `head_benchmark_ms_mean`
 - Meaning: evaluation report for a given pruning rate/config on a fixed checkpoint + val manifest.